HW6_Problem2

January 28, 2026

1 Problem 2

1.0.1 In this assignment, you will implement and train a Generative Adversarial Network (GAN) from scratch using the Fashion-MNIST dataset. Your GAN will learn to generate synthetic fashion item images that resemble real samples from selected categories. Hint: Use tensorflow 2.14 and follow the code of the book.

[ ]:
```
import tensorflow as tf
print(tf.__version__)
```

```
2.18.0
```

1.1 2.1

1.1.1 Load the Fashion-MNIST dataset through tensorflow.keras.datasets, keep the data on the training set, select and retain only the T-shirt/top, Trousers, and Pullover classes, and scale the pixel values to the range [-1, 1].

[ ]:
```
import numpy as np
from tensorflow.keras.datasets import fashion_mnist

# Load Fashion-MNIST (only use training set for GANs)
(X_train_full, y_train_full), _ = fashion_mnist.load_data()

# Keep only classes: 0 = T-shirt/top, 1 = Trouser, 2 = Pullover
selected_classes = [0, 1, 2]
mask = np.isin(y_train_full, selected_classes)

X_train = X_train_full[mask]
y_train = y_train_full[mask]

# Reshape and scale images to [-1, 1]
X_train = X_train.astype("float32") / 127.5 - 1.0  # [0,255] -> [-1,1]
X_train = np.expand_dims(X_train, axis=-1)        # shape: (n, 28, 28, 1)

# Print final shape and class distribution
print(f'Training set shape: {X_train.shape}')
for class_label in selected_classes:
```

```
    count = np.sum(y_train == class_label)
    print(f'Class {class_label}: {count} samples')
```

```
Training set shape: (18000, 28, 28, 1)
Class 0: 6000 samples
Class 1: 6000 samples
Class 2: 6000 samples
```

1.2 2.2

1.2.1 Create a generator that takes input random noise vectors of size 30. It has two hidden layers, each with 100 and 150 units, with an activation function ReLU and a He normal kernel initializer. It outputs a fully connected layer of a 784-dimensional vector with activation tanh, which is reshaped to a 28 by 28 pixels image.

[ ]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Reshape
from tensorflow.keras.initializers import HeNormal

# Define latent vector size
codings_size = 30

# Generator model
generator = Sequential([
    Input(shape=(codings_size,)),
    Dense(100, activation='relu', kernel_initializer=HeNormal()),
    Dense(150, activation='relu', kernel_initializer=HeNormal()),
    Dense(784, activation='tanh'),
    Reshape((28, 28))
])

generator.summary()
```

```
Model: "sequential_14"
```

```
 Layer (type)                Output Shape              ␣
↪Param #

 dense_30 (Dense)            (None, 100)                ␣
↪3,100

 dense_31 (Dense)            (None, 150)               ␣
↪15,150

 dense_32 (Dense)            (None, 784)               ␣
↪118,384
```

```
 reshape_5 (Reshape)                 (None, 28, 28)                ␣
↪  0

 Total params: 136,634 (533.73 KB)

 Trainable params: 136,634 (533.73 KB)

 Non-trainable params: 0 (0.00 B)
```

1.3 2.3

1.3.1 Create a Discriminator that reads a 28 by 28 pixel image and classifies it as "real" or "fake". The Discriminator consists of two hidden layers, each with 150 and 100 units, with an activation function ReLU and a He normal kernel initializer. A sigmoid function activates the output layer.

[ ]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense
from tensorflow.keras.initializers import HeNormal

# Discriminator model
discriminator = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(150, activation='relu', kernel_initializer=HeNormal()),
    Dense(100, activation='relu', kernel_initializer=HeNormal()),
    Dense(1, activation='sigmoid')
])

discriminator.summary()
```

```
Model: "sequential_15"

 Layer (type)                        Output Shape                  ␣
↪Param #

 flatten_5 (Flatten)                 (None, 784)                   ␣
↪  0

 dense_33 (Dense)                    (None, 150)                   ␣
↪117,750
```

```
 dense_34 (Dense)                        (None, 100)                  ␣
↪15,100

 dense_35 (Dense)                        (None, 1)                    ␣
↪101


Total params: 132,951 (519.34 KB)

Trainable params: 132,951 (519.34 KB)

Non-trainable params: 0 (0.00 B)
```

1.4 2.4

1.4.1 Connect the generator and discriminator to set up a GAN pipeline. Use the Binary Cross Entropy as the loss function and the RMSProp optimizer for both the discriminator and the GAN models. Create batches of 32 images by slicing the training set. Train the GAN for 10 epochs, alternating between updating the Discriminator and Generator models on each batch. After each epoch of training ends, visualize 32 generated images.

Set seed for reproducibility

[ ]:
```
import tensorflow as tf

np.random.seed(42)
tf.random.set_seed(42)
```

Compile discriminator and GAN

[ ]:
```
gan = Sequential([generator, discriminator])
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop",␣
 ↪metrics=["accuracy"]) #added accuracy metrics for extra insight
discriminator.trainable = False
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

Prepare the dataset

[ ]:
```
from tensorflow.data import Dataset

batch_size = 32
dataset = Dataset.from_tensor_slices(X_train).shuffle(buffer_size=1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

Define the plotting function

[ ]:
```python
import matplotlib.pyplot as plt

def plot_multiple_images(images, n_cols=8):
    '''Displays a batch of grayscale images in a grid layout.

    Parameters:
    ----------
    images : tf.Tensor or np.ndarray
        A batch of generated images with pixel values in the range [-1, 1].
        Each image should have shape (28, 28).
    n_cols : int
        Number of images to display per row in the grid layout.

    Behavior:
    ---------
    - Automatically rescales the pixel values from [-1, 1] to [0, 1] for␣
 ↪display.
    - Calculates the number of rows based on the number of images and columns.
    - Uses matplotlib to render a grid of images with axes turned off.

    Notes:
    ------
    This function is typically called after each GAN training epoch to visualize
    generated images and monitor the quality of outputs over time.

    '''
    images = (images + 1) / 2  # Rescale from [-1, 1] to [0, 1]
    n_images = len(images)
    n_rows = n_images // n_cols
    plt.figure(figsize=(n_cols, n_rows))
    for index, image in enumerate(images):
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(image.numpy(), cmap="gray")
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Train the GAN

[ ]:
```python
import time
import matplotlib.pyplot as plt

def train_gan(gan, dataset, batch_size, codings_size, n_epochs):
    '''
    Trains a Generative Adversarial Network using a custom training loop.
```

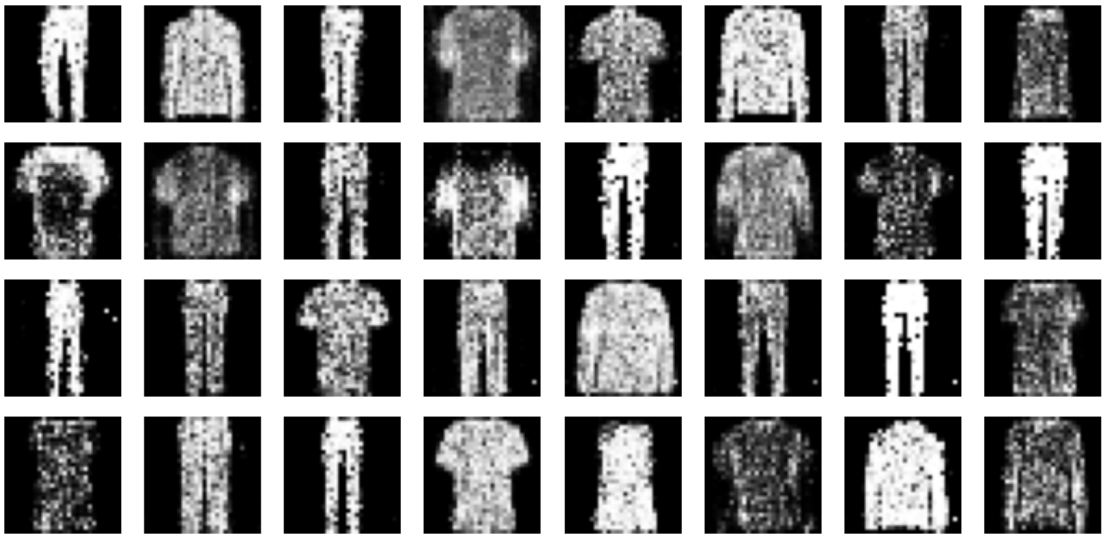
```
    This function alternates between training the discriminator and the␣
↪generator
    on batches of real and synthetic images. After each epoch, it visualizes a␣
↪grid
    of generated images.

    Parameters:
    ----------
    gan : keras.Sequential
        A compiled Sequential model that chains the generator and the␣
↪discriminator.
   dataset : tf.data.Dataset
        A dataset of real training images.
        The images are expected to be scaled to the range [-1, 1] and have␣
↪shape (28, 28, 1).
    batch_size : int
        The number of samples per training batch.
    codings_size : int
        Dimensionality of the random noise vector fed to the generator.
    n_epochs : int
        Number of full passes over the dataset.

    Training Logic:
    ---------------
    For each batch in each epoch:
        - Phase 1: Train the discriminator to distinguish real vs. fake images.
        - Phase 2: Train the generator, via the combined GAN model, to produce
          images that can trick the discriminator.

    Notes:
    ------
    - Before each `train_on_batch()` call, `discriminator.trainable` is toggled:
        - It is set to `True` when training the discriminator.
        - It is set to `False` when training the generator via the GAN model.
    - I decided it would be better not to work in an older Tensorflow version,
      and this manual toggling is necessary in TensorFlow 2.18+ because␣
↪`trainable=False`
      does not always work automatically inside nested models, unless␣
↪explicitly updated at runtime.
    - At the end of each epoch, 32 generated images are visualized using a grid.

    '''
    generator, discriminator = gan.layers
    start_time = time.time()

    for epoch in range(n_epochs):
```

```
        print(f"\nEpoch {epoch + 1}/{n_epochs}")
        epoch_start = time.time()

        for step, X_batch in enumerate(dataset):
            X_batch = tf.squeeze(X_batch, axis=-1)

            # Phase 1 - Train the Discriminator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = generator(noise)
            X_fake_and_real = tf.concat([generated_images, X_batch], axis=0)
            y1 = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)

            discriminator.trainable = True
            d_loss, d_acc = discriminator.train_on_batch(X_fake_and_real, y1)

            # Phase 2 - Train the Generator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            y2 = tf.constant([[1.]] * batch_size)

            discriminator.trainable = False
            g_loss = gan.train_on_batch(noise, y2)

            if step % 100 == 0:
                print(f"  Step {step} - d_loss: {d_loss:.4f}, d_acc: {d_acc:.
↪4f}, g_loss: {g_loss:.4f}")

        epoch_duration = time.time() - epoch_start
        print(f"Epoch {epoch + 1} duration: {epoch_duration:.2f} seconds")

        # Visualization
        print(f"Visualizing generated images after epoch {epoch + 1}")
        noise = tf.random.normal(shape=[32, codings_size])
        generated_images = generator(noise)
        plot_multiple_images(generated_images, n_cols=8)

    total_time = time.time() - start_time
    print(f"\nTotal training time: {total_time:.2f} seconds")
```

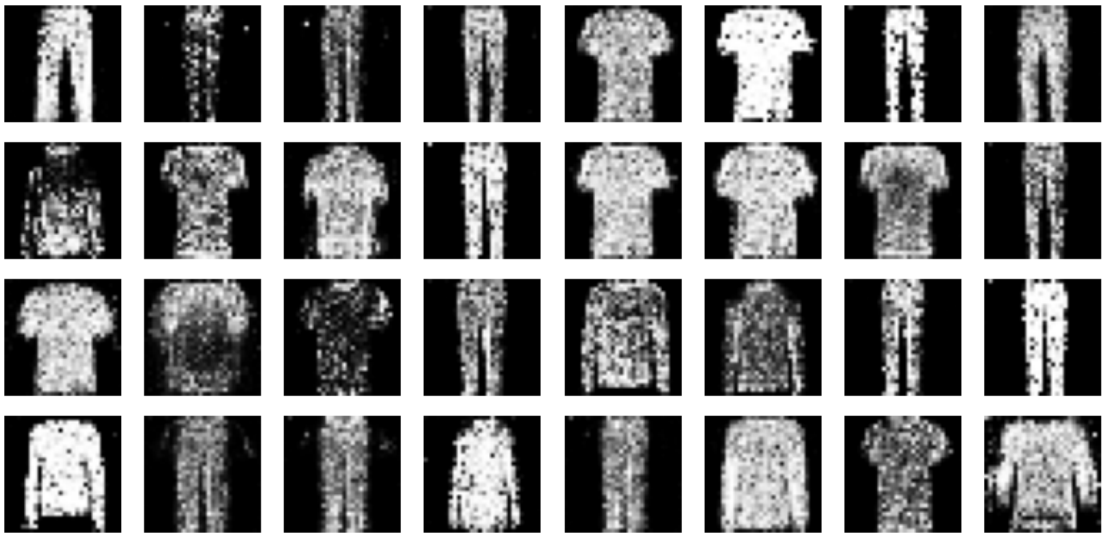
[ ]:
```
train_gan(gan, dataset, batch_size, codings_size, n_epochs=10)
```

```
Epoch 1/10
  Step 0 - d_loss: 0.6661, d_acc: 0.5865, g_loss: 0.8321
  Step 100 - d_loss: 0.6662, d_acc: 0.5864, g_loss: 0.8318
  Step 200 - d_loss: 0.6662, d_acc: 0.5863, g_loss: 0.8317
  Step 300 - d_loss: 0.6662, d_acc: 0.5862, g_loss: 0.8317
  Step 400 - d_loss: 0.6662, d_acc: 0.5863, g_loss: 0.8317
  Step 500 - d_loss: 0.6661, d_acc: 0.5864, g_loss: 0.8319
```

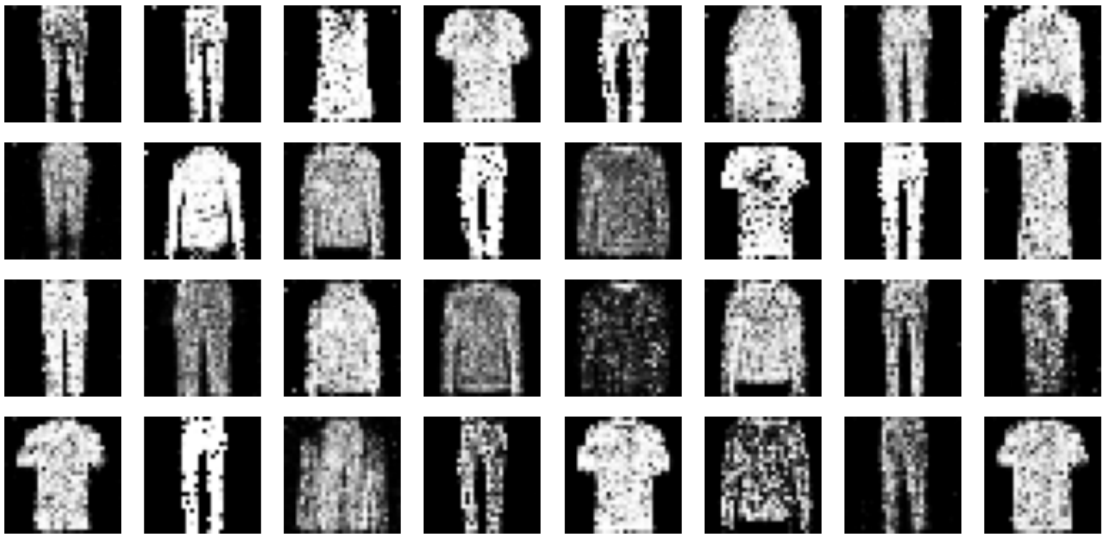
```
Epoch 1 duration: 3.78 seconds
Visualizing generated images after epoch 1
```



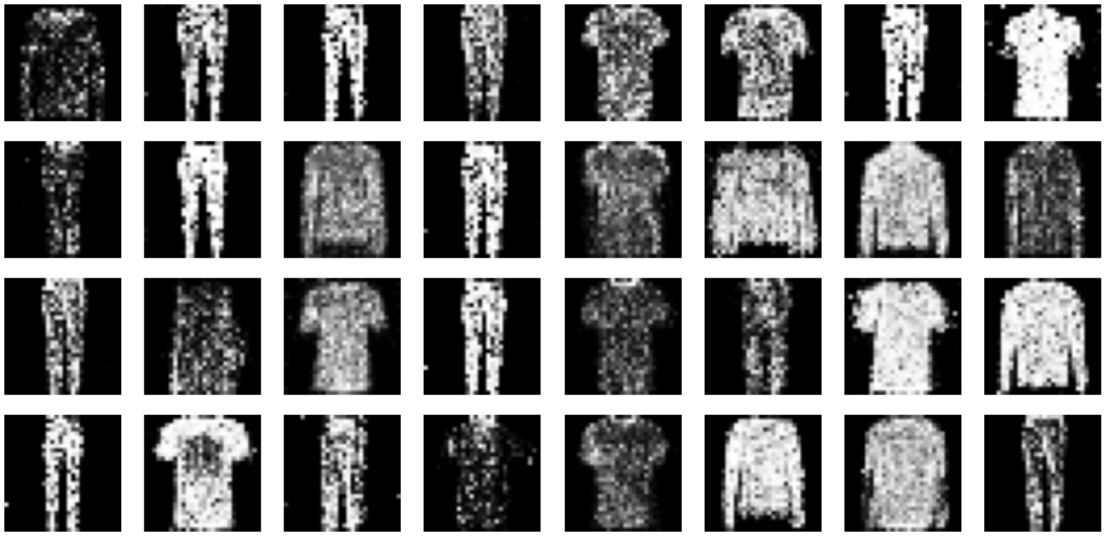
```
Epoch 2/10
  Step 0 - d_loss: 0.6662, d_acc: 0.5863, g_loss: 0.8318
  Step 100 - d_loss: 0.6660, d_acc: 0.5866, g_loss: 0.8321
  Step 200 - d_loss: 0.6660, d_acc: 0.5867, g_loss: 0.8323
  Step 300 - d_loss: 0.6659, d_acc: 0.5867, g_loss: 0.8325
  Step 400 - d_loss: 0.6660, d_acc: 0.5866, g_loss: 0.8325
  Step 500 - d_loss: 0.6659, d_acc: 0.5867, g_loss: 0.8329
Epoch 2 duration: 3.85 seconds
Visualizing generated images after epoch 2
```



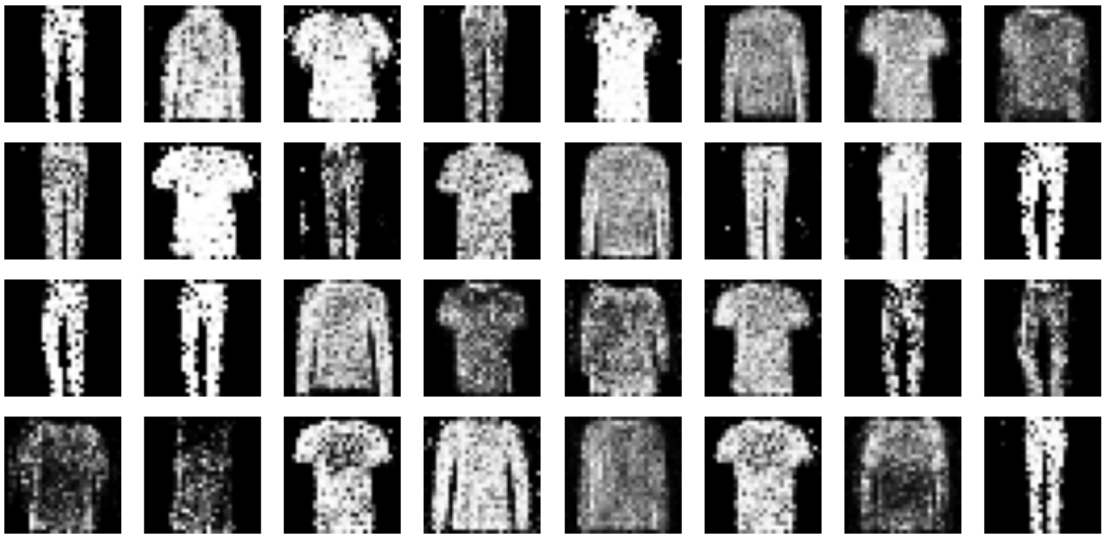
```
Epoch 3/10
  Step 0 - d_loss: 0.6659, d_acc: 0.5868, g_loss: 0.8328
  Step 100 - d_loss: 0.6658, d_acc: 0.5870, g_loss: 0.8331
  Step 200 - d_loss: 0.6657, d_acc: 0.5872, g_loss: 0.8337
  Step 300 - d_loss: 0.6657, d_acc: 0.5871, g_loss: 0.8338
  Step 400 - d_loss: 0.6657, d_acc: 0.5873, g_loss: 0.8340
  Step 500 - d_loss: 0.6656, d_acc: 0.5874, g_loss: 0.8342
Epoch 3 duration: 3.83 seconds
Visualizing generated images after epoch 3
```



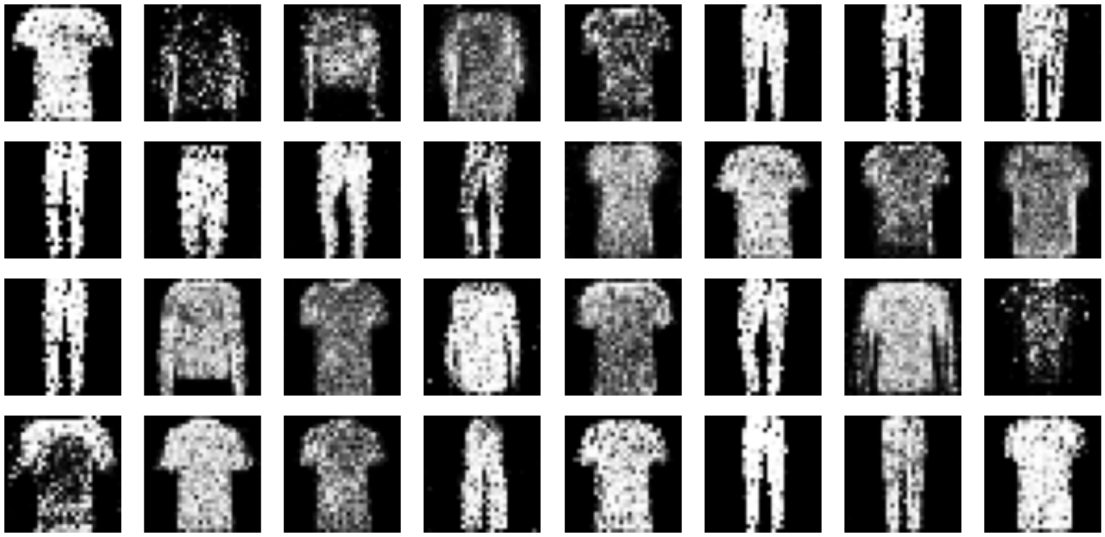
```
Epoch 4/10
  Step 0 - d_loss: 0.6657, d_acc: 0.5874, g_loss: 0.8343
  Step 100 - d_loss: 0.6657, d_acc: 0.5874, g_loss: 0.8345
  Step 200 - d_loss: 0.6657, d_acc: 0.5874, g_loss: 0.8344
  Step 300 - d_loss: 0.6657, d_acc: 0.5875, g_loss: 0.8346
  Step 400 - d_loss: 0.6657, d_acc: 0.5875, g_loss: 0.8349
  Step 500 - d_loss: 0.6655, d_acc: 0.5877, g_loss: 0.8353
Epoch 4 duration: 3.92 seconds
Visualizing generated images after epoch 4
```



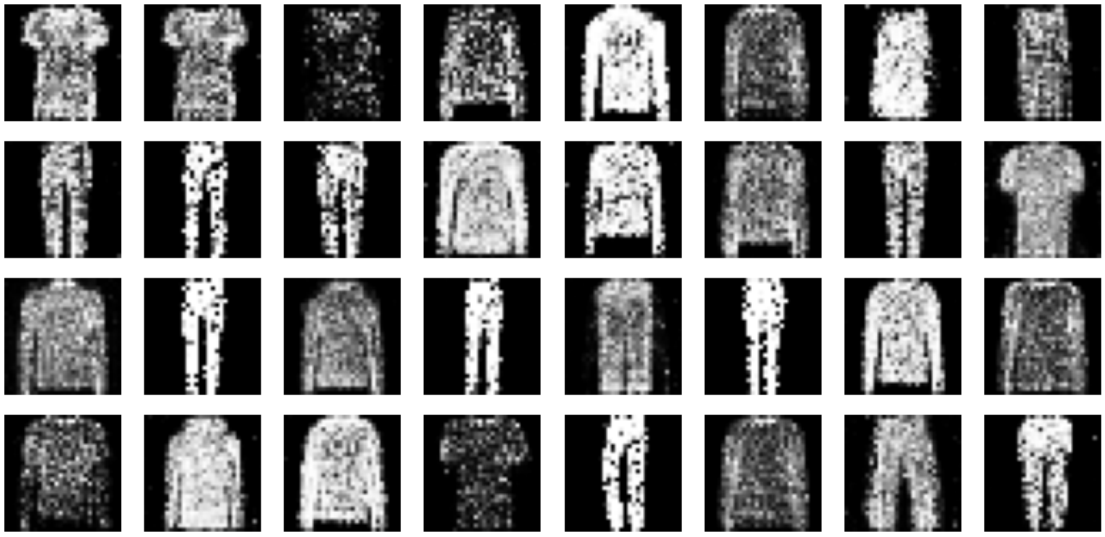
```
Epoch 5/10
  Step 0 - d_loss: 0.6655, d_acc: 0.5877, g_loss: 0.8355
  Step 100 - d_loss: 0.6654, d_acc: 0.5878, g_loss: 0.8356
  Step 200 - d_loss: 0.6653, d_acc: 0.5881, g_loss: 0.8359
  Step 300 - d_loss: 0.6652, d_acc: 0.5883, g_loss: 0.8362
  Step 400 - d_loss: 0.6652, d_acc: 0.5885, g_loss: 0.8366
  Step 500 - d_loss: 0.6652, d_acc: 0.5885, g_loss: 0.8367
Epoch 5 duration: 3.91 seconds
Visualizing generated images after epoch 5
```



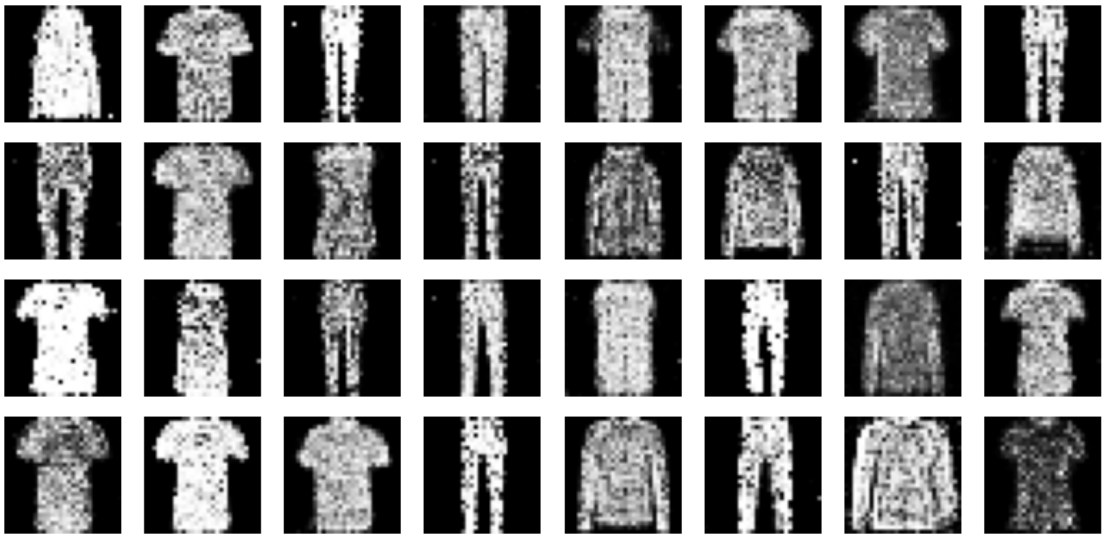
```
Epoch 6/10
  Step 0 - d_loss: 0.6651, d_acc: 0.5886, g_loss: 0.8368
  Step 100 - d_loss: 0.6651, d_acc: 0.5886, g_loss: 0.8370
  Step 200 - d_loss: 0.6651, d_acc: 0.5887, g_loss: 0.8371
  Step 300 - d_loss: 0.6651, d_acc: 0.5887, g_loss: 0.8373
  Step 400 - d_loss: 0.6651, d_acc: 0.5888, g_loss: 0.8375
  Step 500 - d_loss: 0.6650, d_acc: 0.5889, g_loss: 0.8379
Epoch 6 duration: 3.96 seconds
Visualizing generated images after epoch 6
```



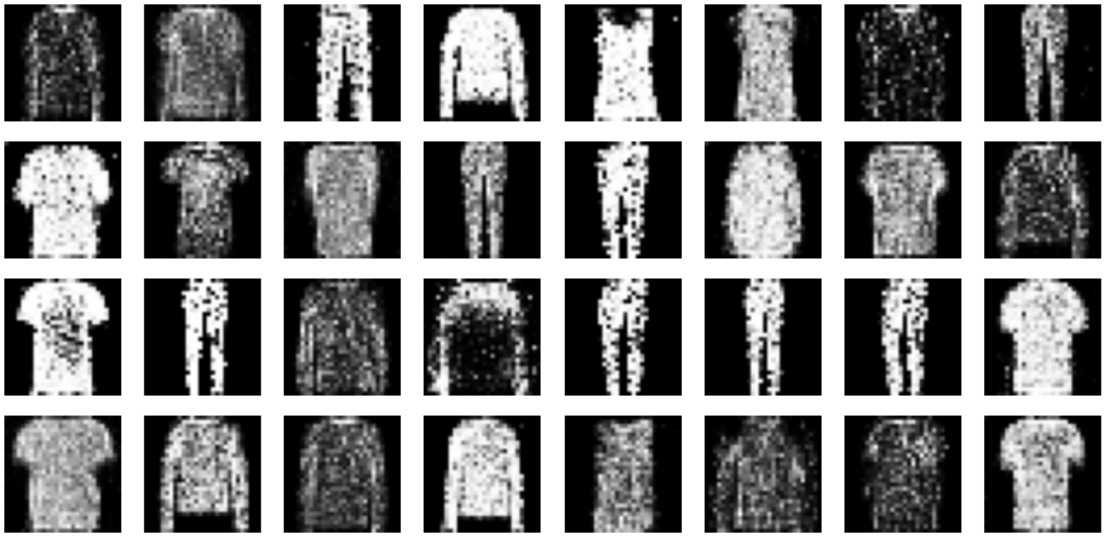
```
Epoch 7/10
  Step 0 - d_loss: 0.6650, d_acc: 0.5889, g_loss: 0.8378
  Step 100 - d_loss: 0.6649, d_acc: 0.5890, g_loss: 0.8380
  Step 200 - d_loss: 0.6649, d_acc: 0.5891, g_loss: 0.8382
  Step 300 - d_loss: 0.6648, d_acc: 0.5892, g_loss: 0.8385
  Step 400 - d_loss: 0.6648, d_acc: 0.5892, g_loss: 0.8387
  Step 500 - d_loss: 0.6648, d_acc: 0.5893, g_loss: 0.8389
Epoch 7 duration: 3.88 seconds
Visualizing generated images after epoch 7
```



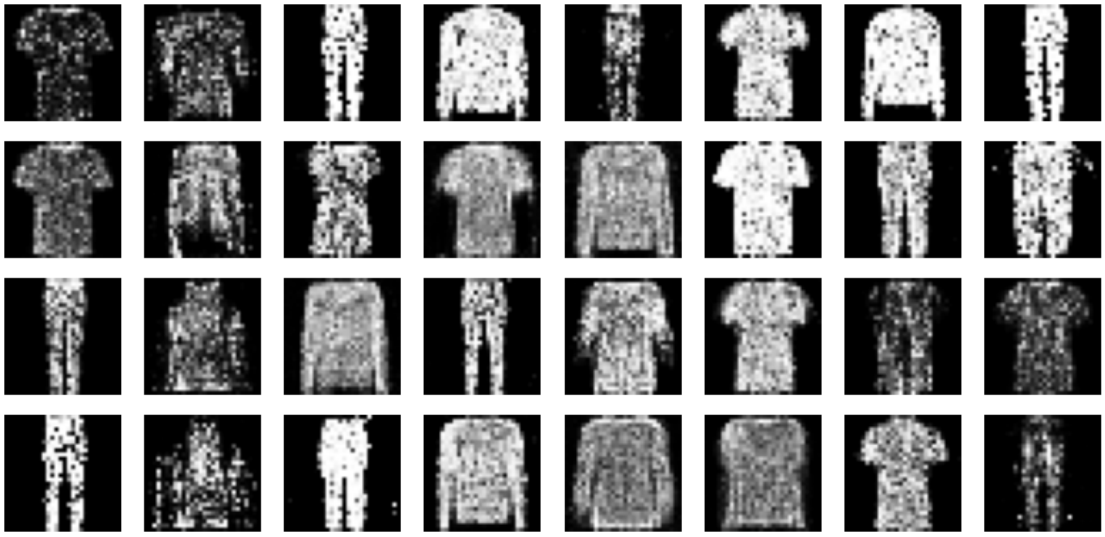
```
Epoch 8/10
  Step 0 - d_loss: 0.6647, d_acc: 0.5893, g_loss: 0.8390
  Step 100 - d_loss: 0.6647, d_acc: 0.5894, g_loss: 0.8392
  Step 200 - d_loss: 0.6647, d_acc: 0.5894, g_loss: 0.8394
  Step 300 - d_loss: 0.6646, d_acc: 0.5896, g_loss: 0.8397
  Step 400 - d_loss: 0.6646, d_acc: 0.5897, g_loss: 0.8400
  Step 500 - d_loss: 0.6645, d_acc: 0.5898, g_loss: 0.8402
Epoch 8 duration: 3.99 seconds
Visualizing generated images after epoch 8
```



```
Epoch 9/10
  Step 0 - d_loss: 0.6645, d_acc: 0.5899, g_loss: 0.8404
  Step 100 - d_loss: 0.6644, d_acc: 0.5900, g_loss: 0.8406
  Step 200 - d_loss: 0.6644, d_acc: 0.5900, g_loss: 0.8408
  Step 300 - d_loss: 0.6644, d_acc: 0.5901, g_loss: 0.8410
  Step 400 - d_loss: 0.6643, d_acc: 0.5903, g_loss: 0.8412
  Step 500 - d_loss: 0.6642, d_acc: 0.5903, g_loss: 0.8416
Epoch 9 duration: 4.01 seconds
Visualizing generated images after epoch 9
```



```
Epoch 10/10
  Step 0 - d_loss: 0.6642, d_acc: 0.5904, g_loss: 0.8417
  Step 100 - d_loss: 0.6642, d_acc: 0.5905, g_loss: 0.8417
  Step 200 - d_loss: 0.6642, d_acc: 0.5906, g_loss: 0.8419
  Step 300 - d_loss: 0.6642, d_acc: 0.5906, g_loss: 0.8419
  Step 400 - d_loss: 0.6642, d_acc: 0.5907, g_loss: 0.8419
  Step 500 - d_loss: 0.6642, d_acc: 0.5908, g_loss: 0.8422
Epoch 10 duration: 3.97 seconds
Visualizing generated images after epoch 10
```



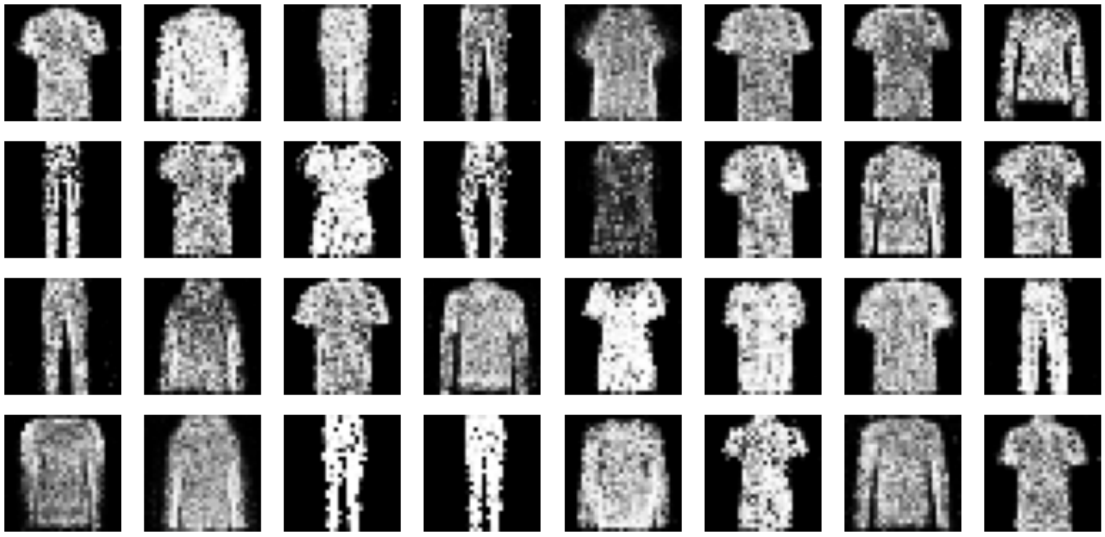
```
Total training time: 43.49 seconds
```

1.5 2.5

1.5.1 After training the Generator, feed it with 32 random noise vectors and visualize the 32 generated images. Are you satisfied with the results?

[ ]:
```
# Generate and visualize 32 new images after training
print("Generated images from trained generator:")
noise = tf.random.normal(shape=[32, codings_size])
generated_images = generator(noise)
plot_multiple_images(generated_images, n_cols=8)
```

```
Generated images from trained generator:
```



The generator has learned some basic structure of the three selected classes, since the generated images resemble clothing items in general shape. The vast majority of the T-shirts, Trousers, and Pullovers are easily identifiable, while others are still fuzzy or ambiguous. This indicates that the GAN has not yet fully converged, and could benefit from more training epochs, batch normalization, or improved weight scaling or regularization.

1.6 2.6

1.6.1 What is the accuracy of the discriminator in predicting that the generated images of the previous question are fake?

[ ]:
```
# Evaluate discriminator's accuracy on the 32 generated images
labels_fake = tf.constant([[0.]] * 32)  # label = fake
loss, accuracy = discriminator.evaluate(generated_images, labels_fake,␣
↪verbose=0)

print(f"\nDiscriminator accuracy on generated (fake) images: {accuracy:.4f}")
```

```
Discriminator accuracy on generated (fake) images: 0.7188
```

The discriminator achieved an accuracy of 0.7188 on the 32 generated (fake) images. This means it correctly identified approximately 23 out of 32 images as fake. This result indicates that the discriminator is still relatively strong, but not perfect — which is a good sign. It suggests that the generator has started producing outputs that are somewhat realistic, to the point that the discriminator misclassifies them roughly 28% of the time. Ideally, in a well-balanced GAN, we aim for the discriminator to have an accuracy around 50%, meaning it can no longer reliably distinguish real from fake. So, while
his result shows some progress, further training or architectural improvements could help the generator produce even more convincing outputs.